Scoring Specification (Deliverable D1)

SBOM Supply Chain Risk Scoring: A Contextual Component Risk Ranking Framework

William A. Davis
CS 690 Graduate Research Project, Fall 2026
Advisor: Dr. Omar Ochoa
Covers Task D (scoring criteria and weights) and Task E (scoring inputs and outputs)
Version 1.0 draft, October 07, 2026

1. Purpose

This specification defines exactly how the pipeline turns a CycloneDX SBOM into a ranked list of software components. It is the document that milestone M1 approves: the vulnerability enrichment work (Task G) and the scoring engine (Task H) will both be built to it, and any change made after approval will be recorded in the change log in Section 10.

Four principles shape the design. First, the model is rule-based and traceable. It uses no machine learning or language models, and every score is a weighted sum of five named factors whose individual contributions appear in the output. Second, only components with at least one known vulnerability are ranked; a component that is merely old or deeply nested never appears on the risk list for those reasons alone. Third, every run is reproducible. Each run fixes an analysis date and stores every raw API response so it can be repeated exactly. Fourth, every run also produces the CVSS-only ranking from the same data, so the contextual model can always be compared against the baseline.

2. Inputs

The pipeline takes one CycloneDX JSON SBOM (specification versions 1.4 through 1.6) and draws on four public data sources.

Input	Source	Used for
CycloneDX SBOM	Ecosystem CycloneDX generator or Syft	Components, versions, package URLs and dependency relationships
Root application	The SBOM metadata component	Starting point for measuring dependency depth
Vulnerability matches	OSV API, queried by package URL and version	Which vulnerabilities affect each component; GHSA-to-CVE aliases
CVSS base score	NVD API v2 by CVE ID, with OSV severity as a fallback	Factor S
EPSS probability	FIRST EPSS API, as of the analysis date	Factor E
Known exploitation	CISA Known Exploited Vulnerabilities catalog; NVD references tagged Exploit	Factor X
Version release date	The package registry (npm, PyPI, Go module proxy, Maven Central)	Factor A

A component's vulnerability set is every OSV record that affects that exact version, excluding withdrawn records, with records that are aliases of one another (for example a GitHub advisory and its CVE) merged into one. A component that has no package URL, or whose ecosystem OSV does not cover, cannot be matched and is reported as "not assessed" rather than as having no known vulnerabilities.

npm sometimes installs the same package version in more than one place in the tree. Components that share a package URL are merged into a single ranked entry, and that entry takes the shallowest depth among the copies.

3. The Five Factors

Each factor is scaled to the range 0 to 1. Severity, exploit probability and known exploitation describe an individual vulnerability; depth and age describe the component.

Factor	How it is calculated	When data is missing
S, Severity	CVSS base score divided by 10, preferring CVSS v3.1, then v4.0, then v3.0, then v2	Use the advisory's qualitative rating (Critical 0.95, High 0.80, Moderate 0.55, Low 0.25); if there is none, use 0.50 and flag it
E, Exploit probability	The EPSS probability for the CVE on the analysis date	Use 0 and flag it (EPSS covers CVEs only)
X, Known exploitation	1.0 if the CVE is in the CISA KEV catalog; 0.5 if NVD lists public exploit code; otherwise 0	Use 0
D, Dependency depth	1 divided by depth, so a direct dependency scores 1.0, depth 2 scores 0.5 and depth 4 scores 0.25	If unreachable, treat depth as the maximum depth plus one and flag it
A, Component age	Days between the version's release and the analysis date, divided by 1,826 and capped at 1.0 (five years or older scores 1.0)	Use 0.50 and flag it

Severity stays in the model because it is the industry default and the basis of the baseline. Exploit probability adds what CVSS does not measure: the likelihood that a vulnerability will actually be exploited in the next thirty days. Known exploitation adds direct evidence that attacks are already happening, or that working exploit code is public. Depth reflects that a direct dependency is more likely to be reachable from the application's own code and can be upgraded by the project itself, while a deeply nested library often needs an upstream release first. Age reflects maintenance lag, since an old pinned version is more likely to carry unpatched problems and harder upgrade paths.

4. The Score

Each vulnerability v in a component c receives a vulnerability score, and the component's risk combines the worst of these with the two component-level factors:

$$VulnScore(v) = w_1 S(v) + w_2 E(v) + w_3 X(v)$$

$$Risk(c) = \max_{v \in c} VulnScore(v) + w_4 D(c) + w_5 A(c)$$

Because the weights sum to 1 and every factor lies between 0 and 1, the risk score also lies between 0 and 1, and reports show it multiplied by 100. The component is scored on its single worst vulnerability rather than on the worst value of each factor taken separately. Taking each maximum independently could pair one CVE's severity with a different CVE's exploit probability and describe a vulnerability that does not exist. The vulnerability that produces the maximum is called the driving vulnerability and is named in the explanation. Ties are broken by the number of vulnerabilities, then by the higher severity, then alphabetically.

Default weights

Weight	Factor	Value	Rationale
w1	Severity	0.30	The largest single weight, because severity still matters most when nothing else is known
w2	Exploit probability	0.25	The strongest predictive signal of real-world exploitation
w3	Known exploitation	0.20	Close to binary, but decisive when present
w4	Depth	0.15	A contextual adjustment that should not by itself outweigh a severe, exploited flaw
w5	Age	0.10	The weakest evidence of the five, so the smallest weight

Together the three vulnerability factors carry 75 percent of the weight and the two context factors carry 25 percent. The weights are stored as configuration rather than in code, so the sensitivity runs described in Section 8 do not require any code change.

Worked example

Consider a direct dependency whose version was released three and a half years before the analysis date and which has a CVE scored CVSS 9.8, with an EPSS probability of 0.94, listed in the KEV catalog. Its factors are S = 0.98, E = 0.94, X = 1.0, D = 1.0 and A = 0.70, giving a risk of 0.294 + 0.235 + 0.200 + 0.150 + 0.070 = 0.949, or 94.9.

Now consider a second component with the same CVSS 9.8 but an EPSS probability of 0.002, no known exploit, four levels deep, and a version six months old. Its risk is 0.294 + 0.0005 + 0 + 0.0375 + 0.0100 = 0.342, or 34.2. A CVSS-only ranking treats these two as tied. The contextual model separates them by about 61 points and can state exactly why.

5. Explanations

Each ranked component receives a one-sentence explanation built from its two largest weighted contributions. The explanation names the driving vulnerability and its CVSS score, states those two factors in plain words with their point values, always states the depth (either "direct dependency" or "N

levels deep via a → b → c" along the shortest path) and the version's age, appends any data-quality flags, and shows the component's baseline rank. An illustrative example, using a made-up package:

#3 example-lib 1.2.3 (score 88.1, baseline #11): driven by CVE-0000-0001 (CVSS 7.2), which has a 0.86 EPSS exploit probability (+21.5) and is in CISA KEV (+20.0); direct dependency; version 5.2 years old; 2 vulnerabilities.

6. Outputs

For each SBOM, a run writes its results to a folder named for the project and the analysis date. The folder holds the ranking as both JSON and CSV, the CVSS-only baseline ranking, a list of components with no known vulnerabilities or that could not be assessed, and a run record containing the analysis date, tool versions, weights, a hash of the input SBOM, the location of the cached API responses, and counts of every data-quality flag.

Each entry in the ranking records the rank and baseline rank and the movement between them; the component's identity (name, version, ecosystem and package URL); the score; the five factor values and their weighted contributions; the driving vulnerability; the full list of matched vulnerabilities with their CVSS, EPSS, KEV and exploit-reference values; the depth and shortest dependency path; the version's release date; any flags; and the explanation sentence.

7. Baseline

The CVSS-only ranking orders components by the highest CVSS base score among their vulnerabilities, using the same vulnerability set and the same severity fallbacks as factor S, with ties broken by the number of vulnerabilities and then by name. Because both rankings are built from identical data, any difference between them is caused by the factors alone.

8. Evaluation Measures

Measure	Definition	Target
Differentiation	How many components in the contextual top ten are not in the baseline top ten; Kendall's τ between the two full rankings	At least 3 of 10 different, in at least 3 of the 4 test projects
Justification	For every component that enters the top ten relative to the baseline, a non-CVSS factor (E, X or D) is one of its two largest contributions	All entering components
Weight stability	Each weight changed by ±0.05 and ±0.10 with the others rescaled to keep the sum at 1 (20 runs per project), each compared with the default ranking	Top-ten overlap of at least 8 of 10 and Kendall's τ of at least 0.80 in 90 percent of runs
Vulnerability density	Vulnerable components at each depth, and the share of total risk held by direct dependencies	Reported without a target

For a project with fewer than ten vulnerable components, "top ten" means all of them.

9. Known Limitations

Depth is a structural measure. It shows how close a library sits to the application, not whether the vulnerable function is ever called; true reachability analysis is outside the scope of this project. PyGoat's requirements file pins every package, so its direct dependencies had to be inferred, as described in the test project document. Go module graphs are shallow by design, which makes the depth factor less discriminating for Go. Finally, EPSS covers only CVE identifiers, so advisories without a CVE receive an exploit probability of zero and are flagged.

10. Approval and Change Log

Date	Change	Approval
October 11, 2026	Version 1.0 draft: five factors, default weights, inputs and outputs	Pending Dr. Ochoa's written approval (M1)